

SEUPD@CLEF: Team 3DS2A on Measuring Time Evolution of IR Systems with Proximity Search and Reranking Models

Andrea Bruttomesso¹, Daniele Cavazza¹, Alessandro Corrò¹, Simone Peraro¹ and Davide Seghetto¹

¹University of Padua, Italy

Abstract

This report gives an overview of the system developed by Team 3DS2A for Task 1 of the LongEval Lab at CLEF 2025. The team members are students enrolled in the Computer Engineering master's program at the University of Padua. The team developed an information retrieval system tailored to run on a French-language document corpus. The system was evaluated over a nine-month timespan to assess its robustness in terms of precision and recall over time. Throughout development, multiple techniques were explored, including alternative text analyzers, proximity search, chunk-based indexing, and semantic reranking using sentence embeddings. This report presents the system architecture, the experimental strategies adopted, and the performance achieved during the training phase.

Keywords

CLEF, LongEval 2025, Information Retrieval, Search Engine, Reranking, Word Embeddings

1. Introduction

In today's digital age, online searching has become an integral part of daily life, with billions of users worldwide relying on search engines to quickly and accurately find the information they need. A search engine (SE) is a software system designed to fulfill this demand by processing user queries that express specific information needs. However, as the number of web pages continues to grow rapidly, search engines face increasing challenges, one of the most significant being a decline in performance over time. To address this issue, the LongEval Lab, organized by the Conference and Labs of the Evaluation Forum (CLEF), encourages the development of temporal information retrieval (IR) systems capable of adapting to the evolving nature of online content. This work reports on the proposed solution implemented by the 3DS2A group at the University of Padua as part of the Search Engines Course.

Alongside the traditional search pipeline, the system also utilizes more sophisticated techniques like chunk-based search, pseudo-relevance feedback, and reranking based on a sentence embeddings model to enhance the retrieval capability of the queries used.

The paper is organized as follows: Section 2 briefly introduces some related works for past LongEval tasks at CLEF 24; Section 3 describes our approach; Section 4 explains our experimental setup; Section 5 discusses our main findings; finally, Section 6 draws some conclusions and outlooks for future work.

2. Related Work

The LongEval 2025 task follows the same general structure as previous CLEF LongEval editions, with the key difference that this year's training set spans a significantly longer temporal window.

"Search Engines", course at the master degree in "Computer Engineering", Department of Information Engineering, and at the master degree in "Data Science", Department of Mathematics "Tullio Levi-Civita", University of Padua, Italy. Academic Year 2024/2025

✉ andrea.bruttomesso.1@studenti.unipd.it (A. Bruttomesso); daniele.cavazza@studenti.unipd.it (D. Cavazza);
alessandro.corro.1@studenti.unipd.it (A. Corrò); simone.peraro.1@studenti.unipd.it (S. Peraro);
davide.seghetto@studenti.unipd.it (D. Seghetto)



© 2025 Copyright for this paper by its authors.

Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

In past editions, a wide range of information retrieval techniques have been explored, with sentence embedding-based approaches playing a prominent role, especially in the re-ranking phase [1] [2]. For example, Basaglia et al. [3] adopted a reranking strategy based on sentence embeddings, demonstrating its effectiveness in improving retrieval performance on temporally dynamic datasets.

While the discussion about whether stemming or lemmatizing achieves better performance is still open [4] [5], another common technique involves the expansion of query terms [6], and the use of Named Entity Recognition [7].

Our objective is to explore some of these techniques, combining them into new approaches while participating at the LongEval Web retrieval task, using as reference code the one proposed during the Search Engines course at the University of Padua in the academic year 2024/25.

3. Methodology

This section describes the methodologies adopted by our IR system for this task. To build our IR system, we used Apache Lucene 10.1.0 [8], an open source search engine library developed in Java, known for its high performance and rich feature set. Lucene provides a robust framework for indexing and retrieval, which we extended with a reranking module based on sentence embeddings.

Our goal was to enhance the effectiveness of an approach explored in the previous year by Basaglia et al. [3]. We used a similar approach based on reranking a small chunk of documents, but using a larger and more expressive sentence embedding model to capture deeper semantic similarities between queries and documents.

The main components of our system are:

- **WebDocumentParser** to read and parse a *.json* file into a **WebDocument** object.
- **WebAnalyzer** to analyze the contents of a document and return tokens ready to be indexed.
- **DirectoryIndexer** to manage the opening, parsing and indexing of the contents in a specific directory.
- **Searcher** to load the provided queries and perform the matching between those and the indexed documents.

The following diagram illustrates the primary elements of a search engine, highlighting the distinction between offline and online components.



Figure 1: SE architecture

3.1. Parsing

The first step in IR systems is about collecting the documents and provide them to the system in a "cleaned" and organized way. For this purpose, two classes have been developed in our system: the **WebDocument** and **WebDocumentParser** classes, which are responsible for reading the document and abstracting them in an organized Java object. We developed our parser class to read through the *.json* file format using *Jackson XML Parser*, and convert them into the specific Java class *WebDocument*.

The structure of the *.json* files is rather simple, and it is made as an array of anonymous elements with only two fields: *id* and *contents*, therefore the associated Java class will have the same structure. However, while running the system, we discovered that the LongEval team has provided the queries results with a different convention of the *[id]* field, and we had to remove the prefix "doc" from the field representing the document id. The most important thing about the parsing process is that it also provides a pre-processing step to clean the contents of the document. This step does not aim to perform detailed text analysis, as that responsibility is delegated to the analyzer component, therefore we just focus on the removal of html tags, of unwanted punctuation, of URLs and normalization of consecutive whitespaces.

All of this happens while accessing the list of documents in the *.json* file, so the process is performed only when actually accessing the required document when iterating the collection, rather than processing the whole collection at a single time. The **WebDocumentParser** provides access to the documents contained in a single file through the canonical iteration methods and returns the **WebDocument** object ready to be analyzed by the consumer **WebAnalyzer**.

3.2. Analyzing

Once we have access to the contents of our document, we must extract a sequence of textual elements - called *tokens* - to be provided to the indexer. These tokens represent the basic units upon which the search process operates. The main purpose of the analyzer component is to provide those tokens and apply some filtering and optimizations to improve the performance of our system.

To perform these operations, we developed our implementation of analyzer, called **WebAnalyzer**, which provides access to tokens through the **createComponents** method. The pipeline to generate a token is divided into several steps. First of all, we want to generate tokens from the text using one of the provided Lucene classes, like the **WhitespaceTokenizer**, the **LetterTokenizer** or the **StandardTokenizer**. Using one of these allows us to split the text into words or other components, which we can consequently analyze. After applying a **LowerCaseFilter** component to the token, to normalize the capitalization of the text, we want to remove the tokens which may not be relevant for our purpose: we use a **LengthFilter** to remove too short or too long tokens, and finally a **StopFilter** component to eliminate the stopwords. In order to make the text more uniform, we also apply an **ElisionFilter** and **ASCIIFoldingFilter** components to remove elisions and regularize uncommon characters that may be present in the text, just before filtering the stopwords.

The last and probably most important step in our analysis pipeline is about the stemming process. We want our system to be able to match terms that might not be exactly written in the same way in the documents and queries, like (e.g. plural/singular nouns). Therefore, we apply a stemmer component at the end of the pipeline. Since the stemming process is a very delicate part of the analysis, we tried different configurations of the stemmer component, like the **Krovetz**, **SnowBall** and **Porter** stemmers, but also the Lucene's **FrenchLight/FrenchMinimal StemFilter**, since our collection is made of French queries and documents. At the end of the process, the **WebAnalyzer** generates a sequence of tokens ready to be consumed by our indexer.

To ensure flexibility and allow for easy experimentation with different analysis strategies, the **WebAnalyzer** component is configurable using an external XML file. This file specifies the parameters used in the analysis pipeline, such as the tokenizer type, minimum and maximum token length, the stemmer to be applied, and the activation of specific filters such as **ASCIIFoldingFilter** or **ElisionFilter**. The configuration is defined using an **AnalyzerParams** class, which is populated from the XML using

Jackson annotations.

3.2.1. POS Tagging and Lemmatization

In order to improve the matching performance of our system, we tried to implement different versions of the analysis component. One of the first ideas was to take advantage of the OpenNLP [9] project to reduce the number of indexed terms and improve search results by expanding some types of words. In order to implement this kind of configuration, we tried to implement an **OpenNLPAalyzer** class using OpenNLP models for sentence detection, tokenization, POS-tagging, and lemmatization. We also developed a `POSTagFilter Tokenstream` component to discard unwanted types of tokens. However, when trying to perform the analysis of the documents, we couldn't achieve a sufficient execution time from the system, and therefore we dropped this implementation.

A second attempt at lemmatization was made using a more simple model based on Leff [10][11], a large-scale morphological and syntactic lexicon for French. In this case, we statically generated a copy of the collection documents containing the lemmatized terms, and then we used this collection as input to our system, but removing the stemming process from the analysis pipeline. In this case, the running time was very efficient, but we soon discovered that the lemmatization process didn't improve the matching performances of our system. Therefore, we dropped also this lemmatization process.

3.3. Indexing

Upon receiving the tokens generated from the analyzer's pipeline, the system must store the terms in the index. Lucene stores an inverted index, associating each term with a list of postings, that is the list of document ids containing that index term. In our configuration to improve the search capabilities of our system, we configured the index to also store the term frequencies and relative positions so that we could take advantage of the *tf-idf* metrics of each term and of phrase search. We also implemented a more complex indexing procedure, where each document is split into chunks, in order to improve the precision during the search phase, as better described in 3.3.1.

To manage the indexing process, we developed the **DirectoryIndexer** class, which allows us to parse and index an entire directory containing *.json* files, especially useful since in our case we are dealing with a sequence of "snapshot" directories, one for each month. Moreover, an efficient **MultiThreadIndexer** class has been developed, so that multiple *.json* files are indexed at the same time.

3.3.1. Chunk Indexing

In order to enhance the retrieval performance of our system, we also tried an approach at index-time. Chunk indexing is a strategy designed to enhance retrieval granularity by dividing documents into semantically coherent text segments - called *chunks* - before indexing. The effectiveness of chunk-based approaches has been highlighted in both neural and classical IR frameworks, in particular, Yin and Schütze[12] operated on multiple levels of text granularity to improve matching performance across chunks. Their findings support the idea that representing and comparing textual content at different granular levels leads to improved relevance estimation.

Inspired by this principle, our system implements chunk indexing with overlapping windows of 10 sentences, as follows:

1. **Sentence-based Segmentation:** Documents are segmented into chunks using the `fr-sent.bin` French sentence model provided by OpenNLP[9]. Sentences are then grouped in chunks of 10 sentences each.
2. **Sliding Window with Overlap:** To preserve semantic continuity across chunks, a sliding window approach is used. Each new chunk begins after reusing the last m sentences (3 in this case) of the previous chunk as overlap.

3. **Unique Identification:** Each chunk is indexed as an independent Lucene document with a unique identifier formatted as `<docID>_<chunkIndex>`, ensuring traceability to its source document.

During retrieval, the system operates at the chunk level, but the results are aggregated at the document level using a simple yet effective post-processing step:

- For each original document, only the highest-scoring chunk is retained.
- The final score assigned to the document corresponds to the score of this best chunk, thus reflecting its most relevant passage.

This strategy enables both fine-grained relevance matching and efficient document-level ranking, particularly beneficial in multilingual or domain-diverse collections. Further considerations on the impact of chunking on retrieval performance are discussed in Section 5.8.

3.4. Searching

Moving on with the "online" section of our system, the *Searcher* class is the component tasked with interpreting user queries and scanning indexed documents to identify the most relevant matches. In order to perform the searching process, we also developed a *TopicParser* class, which is responsible to parse the queries file and pass them with their id to the searcher, in a similar way to what we have seen when parsing the *.json* documents. Several components have been tried to further improve the search phase, where the actual matching is performed through the **BM25** model.

3.4.1. BM25

The next step involves retrieving relevant documents based on the queries submitted. This process entails identifying the documents that match most closely to each query, using scoring functions to evaluate their relevance. Each document is assigned a score and the results are ranked accordingly, from highest to lowest, under the assumption that the highest scoring documents are more relevant to the query. Among these scoring methods, the BM25 ranking function, part of the "Best Match" (BM) family of retrieval models, stands out. Despite its simplicity, BM25 remains highly effective and continues to perform competitively against more modern retrieval techniques.

3.4.2. Queries

The queries provided by the LongEval team consist of a set of files, one for each monthly snapshot provided. It is possible to work with two different formats, *.trec* files or *.txt* files. We chose to work with *.txt* files for simplicity, but a parser was developed for each version in order to generate the corresponding **QualityQuery** object required by Lucene.

Query terms are then analyzed and used to generate more powerful queries, using different techniques to improve the search result. In the following subsections we describe our implemented procedures.

3.4.3. Fuzzy Search

We also experimented with a fuzzy search extension after observing a high incidence of typographical errors in user queries, like "*amurerie*", which should be "*armurerie*". By replacing exact term matching with a Levenshtein-based fuzzy query, we aimed to recover relevant documents despite misspellings. However, contrary to our expectations, the fuzzy approach consistently underperformed compared to strict matching: the relaxation of edit distance constraints introduced substantial noise and lowered precision, ultimately degrading overall retrieval effectiveness. This surprising outcome led us to favor the original exact-matching pipeline for the final system configuration.

3.4.4. Pseudo-Relevance Feedback

Pseudo-Relevance Feedback (PRF) is an unsupervised technique used to improve retrieval effectiveness by automatically reformulating the query. Unlike traditional relevance feedback, it does not rely on human interaction. Instead, PRF expands the user's query by leveraging terms from top-ranked documents retrieved during an initial search. The method assumes that the most relevant terms can be extracted from the top- k results and used to enhance recall.

Our implementation follows these main steps:

1. **Initial Retrieval:** Execute the original query using BM25 to retrieve the top k documents.
2. **Term Extraction:** For each of the top documents, tokenize the text using the same analyzer as during indexing. Filter out stopwords and tokens with inappropriate length. Then, compute term frequency (TF) and document frequency (DF) per term.
3. **Scoring and Ranking:** Compute a BM25-inspired weight for each candidate expansion term:

$$\text{score}(t) = \text{TF}(t) \cdot \log \left(1 + \frac{N - \text{DF}(t) + 0.5}{\text{DF}(t) + 0.5} \right)$$

where N is the total number of documents and $\text{DF}(t)$ is the document frequency of term t .

4. **Query Expansion:** Select the top- n scoring terms and construct a new query combining:
 - The original query, boosted by a weight α
 - The expansion terms, boosted by a lower weight β
5. **Final Retrieval:** Submit the expanded query to obtain the final re-ranked list.

Formally, the final query Q' is built as:

$$Q' = \alpha \cdot Q + \beta \cdot \sum_{t \in T_{\text{exp}}} t$$

where T_{exp} is the set of top expansion terms.

3.4.5. Proximity Search

Proximity search enriches traditional term-based retrieval by introducing a measure of how closely query terms co-occur within a document, which often signals a stronger semantic relationship. Documents in which the full set of query terms appears within this slop window receive a score boost, reflecting their contextual cohesion. For example, the query "red house brick" could be used to retrieve documents that contain phrases like "red house built with bricks", while in the meantime avoiding documents where the words are scattered or spread across. However, as previously mentioned, the documents must still contain all query terms to match the full proximity constraint. So to further capture cases where only subsets of the query are present, we automatically generate every pair and triplet of query terms and apply the same proximity criterion to each combination. We limit the subsets to pairs and triplets to avoid exceeding Lucene's BooleanQuery clause limit of 1024, which would otherwise be reached quickly with longer queries. These proximity constraints are added as optional clauses in the final query, ensuring that documents with locally clustered terms are boosted in rank. Later, we will discuss the tuning of the slop parameter and how its value can influence system performance.

3.4.6. Synonyms

In order to try to improve the performance, the system implements a query expansion technique with synonyms, exploiting the semantic lexicon **WOLF** (WordNet Libre du Français) [13]. This technique allows words semantically similar to the original term to be included in the query, increasing the probability of retrieving relevant documents even if expressed with different terms.

During the Lucene query construction, each token is analyzed and, if the expansion with synonyms is enabled, the WolfManager module is queried. This module manages a dictionary of synonyms extracted from the XML WOLF resource. For each token, if synonyms are present, a `SynonymQuery` is created that includes both the original term and its synonyms. This query is then added to the global Lucene query using the `BooleanQuery` builder, with the `SHOULD` operator, to expand the coverage without penalizing the results based on the original term.

3.4.7. Reranking

To improve the quality of the results, the first k retrieved documents are subjected to a semantic re-ranking phase. The value of k will be discussed in Section 5.9. For each document, the textual content and the query are sent to a **local server**, which hosts a `SentenceTransformer` model: `all-roberta-large-v1` [14], available on Hugging Face [15]. The model is built upon the RoBERTa-large architecture, a robustly optimized variant of BERT, and is fine-tuned on over one billion sentence pairs to produce semantically meaningful embeddings. Given an input, the model maps it into a dense 1024-dimensional vector space such that semantically similar sentences are located close together. This makes it well-suited for tasks like reranking, semantic search, and clustering.

The server computes these steps:

- It employs the model, which performs a vector representation (embedding) of both the query and the document and evaluates their similarity through the scalar product.
- The score obtained, between 0 and 1, reflects the semantic relevance between the two texts so it's **weighted** and **added** to the original Lucene score, in order to obtain a new ranking value.
- The documents are then reordered based on this aggregated score.

This approach allows us to combine the efficiency of traditional retrieval with the generalization and semantic understanding capabilities offered by Deep Learning models. The re-ranking server is built in Flask [16] and provides an HTTP endpoint (`/evidence-score/d1`) that receives a POST request containing the document and the query. The `SentenceTransformer` model computes the embeddings and returns a semantic similarity score as a JSON response.

3.5. Running the system

Once our components have been developed, we implemented a main **SystemRunner** class to coordinate the entire pipeline execution. The indexing and searching parts can be executed at different times, to simulate the *offline* and *online* deployment phases. The parameters of the system are passed to the components through constructors and *XML* configuration files, in order to improve the flexibility of the system.

The final complete overview of the system is shown in Figure 2.

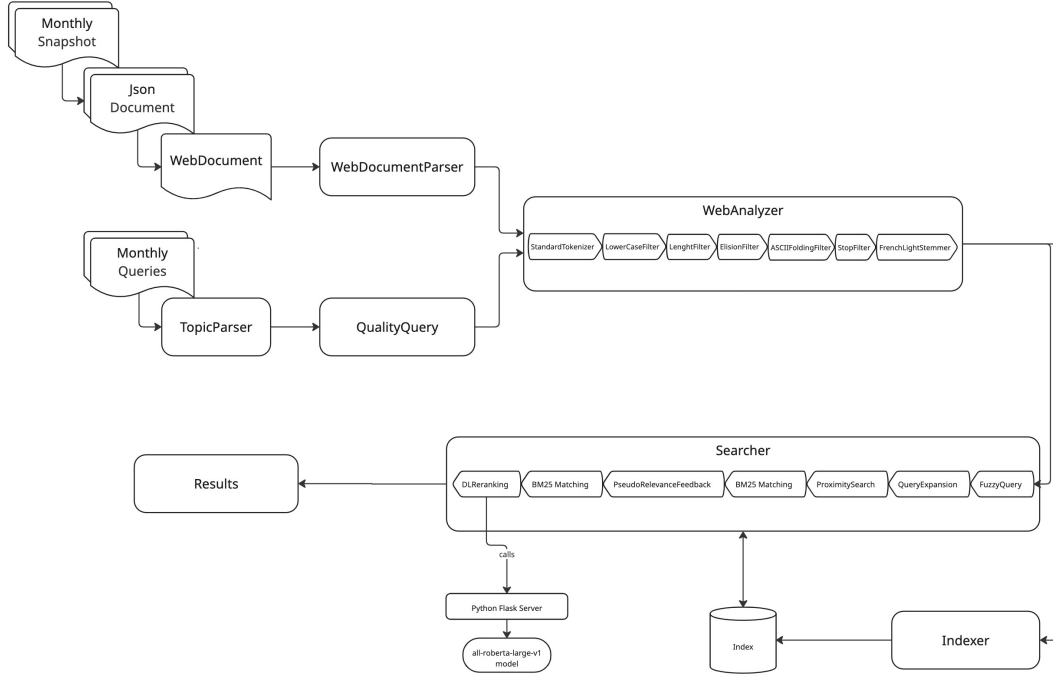


Figure 2: General overview of a possible configuration of our implemented system. We can see how the analyzer is a key element used both in the offline and online phases, to analyze documents and queries. The searcher uses a python flask server to retrieve the score for each document using a LLM model and rearranges the results accordingly. The chunk indexing process is not highlighted here.

4. Experimental Setup

As already discussed, the system has been developed in Java through the Apache Lucene Library, but a few side components have been developed in Python. The source code can be found at <https://bitbucket.org/upd-dei-stud-prj/seupd2425-3ds2a/src/master/>, and as an evaluation measure, we used the metrics provided by the `trec_eval 9.0.7` tool, available at the https://trec.nist.gov/trec_eval/repository.

The system has been run on the collection provided by the LongEval team at <https://clef-longeval.github.io/data/>. We used our personal computers to test our system, where the most expensive runs have been completed using the hardware described in Table 1.

Table 1

Most powerful hardware available

	CPU	RAM	GPU
System 1	i7-10750H	16GB	GTX 1060-Ti
System 2	Ryzen 9 3900x	32GB	RTX 3070

4.1. Training data

The training collection is made up of several components. LongEval team has given us a timeline sequence of documents, extracted from the Qwant search engine through a sequence of months, from June 2022 to February 2023, with every month in a specific subfolder. For every monthly snapshot, the corresponding list of queries to submit has been provided at <https://github.com/clef-longeval/clef-longeval.github.io/tree/master/collection>. Moreover, to understand how the system performed each month, a corresponding *qrels* file has been provided for each month.

Please notice that, in order to use those *qrels* files with the *trec_eval* executable, it is necessary to remove duplicate entries in the *qrels* files, and for this purpose, we developed **QrelsFilter.java** class, which will create a filtered version of the file.

4.2. Evaluation with training data

The evaluation metrics have been computed using the *-m all_trec* parameter of the *trec_eval* executable, which will give us several metrics to examine. For training data, we mostly focused on the **nDCG** and **mAP** metrics, in order to improve the overall result of the system.

5. Results and Discussion

The first thing we need to do to improve our system is to set a reference score for our metrics. In order to do that, we configured our system to use only the main components of Lucene. Upon establishing a baseline evaluation score, we could then try to develop new strategies to improve the final result of the system.

5.1. Baseline performance using Lucene library

To set a baseline for our next runs, we used a simple configuration of the *Indexer*, *Analyzer*, and *Searcher* components, as described in Table 2.

Table 2

Baseline system's configuration. The indexing process doesn't use the chunk-based approach.

	used components from Lucene
Tokenizer	StandardTokenizer
Text normalization	LowercaseFilter, ElisionFilter, ASCIIFoldingFilter
Filtering	LengthFilter(3-100) and StopFilter (French and English)
Stemming	FrenchLightStemmer
Matching model	BM25

When running this base system on the provided training dataset, we achieved very different results depending on the considered month. We repeated the measurements several times and came to the conclusion that some months have very difficult topics or strange judgments, but probably also because some documents have different languages and may not be processed in an efficient way. Nevertheless, we achieve a complete overview of the baseline system's performance in the Table 3.

Table 3

Baseline system's performance. Notice how the score rises from June to January.

Month	mAP	nDCG
2022-06	0.1174	0.2076
2022-07	0.1208	0.2089
2022-08	0.1325	0.2168
2022-09	0.1228	0.2062
2022-10	0.2078	0.3104
2022-11	0.2122	0.3136
2022-12	0.2192	0.3244
2023-01	0.2232	0.3277
2023-02	0.2019	0.2969

5.2. Stemming against Lemmatizing

Even if we couldn't manage to achieve an efficient system using OpenNLP for lemmatizing the documents, we tried a simpler way to generate a lemmatized version of the collection. Using a dictionary-based lemmatizer [10], we generated a lemmatized copy of the collection, including the queries, and fed them to our system, taking care of disabling any active stemmer. The results are depicted in Figure 3 and Figure 4. The comparison with the baseline system highlights how the performance is pretty much aligned with the stemmed version of the collection, but it is important to say that the lemmatizer used is probably too simple. Nowadays our systems could take more advantage of LLM models, and from a better lemmatization it would be possible to further expand each term to improve the final result of the system.

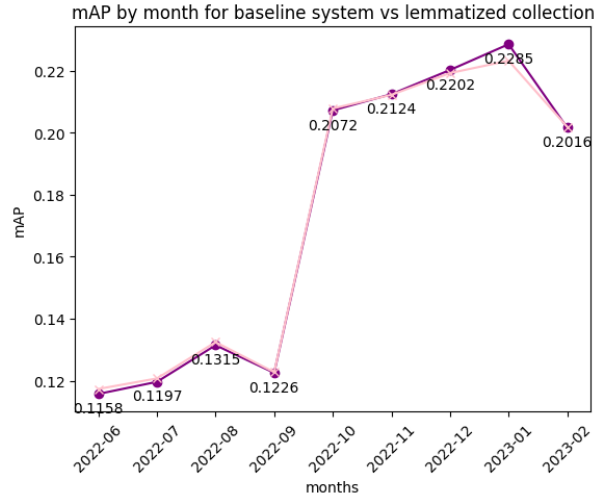


Figure 3: mAP score of the baseline system over the original collection against the lemmatized version of the collection. Reported numbers are the scores obtained using lemmatized collection.

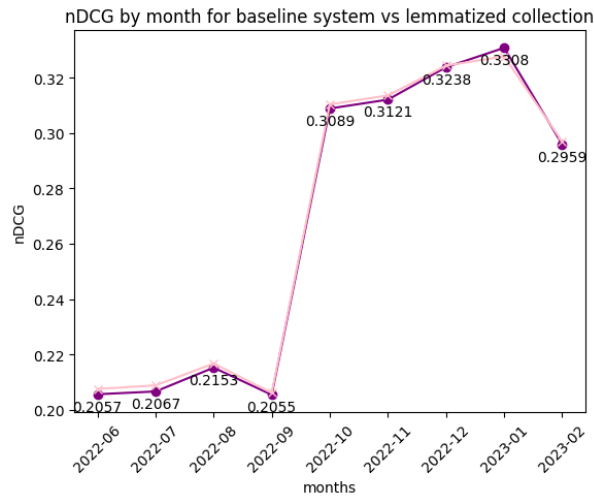


Figure 4: nDCG score of baseline system over the original collection against the lemmatized version of the collection. Reported numbers are the scores obtained using lemmatized collection.

5.3. Choice of stemmer

To assess the effectiveness of different text analysis strategies, we performed a series of experiments combining various tokenizers (Standard, Letter, and Whitespace) with multiple stemmers (FrenchLight, FrenchMinimal, Snowball, and None). All configurations were tested on the 2022-06 dataset.

The evaluation focused on two main retrieval metrics: Mean Average Precision (mAP) and nDCG. Across the board, the FrenchLight stemmer consistently delivered the best results, particularly when paired with the StandardTokenizer, achieving the highest mAP (0.1174) and competitive nDCG scores. Configurations using the LetterTokenizer also performed well, providing strong baseline alternatives.

Interestingly, the use of stopwords filtering did not lead to performance improvements. This trend was consistent across both French and English stoplists, suggesting that aggressive stopwords removal may be useless in this context.

Despite the fact that stopwords filtering did not improve retrieval performance in our current experimental, this feature will remain part of the default analysis pipeline. There are several reasons behind this decision. First, stopwords removal improves robustness by eliminating high-frequency, low-content terms that might introduce noises. Second, stopwords can significantly affect index size and efficiency, especially when dealing with large corpora. Lastly, including stopwords filtering maintains compatibility

with standard IR practices and facilitates future integration of more advanced techniques, such as query expansion or relevance feedback, which often employ a cleaner input signal. Some interesting results can be seen in Table 4.

Table 4

Some configurations on 2022-06

Configuration	mAP	nDCG
Standard+FrLight	0.1174	0.2076
Standard+FrMin	0.1159	0.2057
Standard+Snow	0.1157	0.2060
Standard+Porter	0.1153	0.2054
Standard+Krow	0.1140	0.2045
Letter+FrLight	0.1167	0.2078
Whitespace+FrLight	0.1093	0.1942
No stopwords	0.1175	0.2076

5.4. BM25 function

To evaluate the impact of the parameters of the ranking function on retrieval performance, we experimented with different configurations of the BM25 similarity function, which is the default scoring method used in many modern IR systems.

We evaluated multiple parameter settings of BM25, particularly focusing on variations of the parameters k_1 (term frequency saturation) and b (document length normalization). The baseline run used the default Lucene settings, while two alternative runs explored more aggressive and conservative configurations.

Among the settings tested, the default setting (with parameters $k_1 = 1.2$ and $b = 0.75$) achieved the best overall performance. This setup outperformed alternative configurations, including more aggressive ones such as BM25(2.0, 1.0), which, although slightly better in recall, showed less consistent results and increased noise in top-ranked documents. In contrast, the more conservative configuration BM25(0.9, 0.4) significantly underperformed in both early and deep precision. These results suggest that the default BM25 function offers the best trade-off between precision and robustness for our dataset, and thus was selected as the standard configuration in our system.

5.5. Synonyms

In this section, we are going to talk about the results achieved through synonyms query expansion. The experiment setup was the one used in the baseline. The synonyms included in the queries are analyzed with the same analyzer used for indexing the documents and, furthermore, are assigned a weight of 0.5 to reduce their significance compared to the words originally present in the queries.

As shown in Table 5, query expansion with synonyms did not lead to any performance improvements. In all months analyzed, the mAP and nDCG scores are slightly lower than those of the baseline. This suggests that including synonyms, even with a reduced weight, tends to introduce noise rather than add value to the queries. Based on these results, we decided not to adopt this strategy in future configurations.

5.6. Proximity Search (PS)

Here we present the results obtained through the implementation of proximity search. As anticipated before, the tuning of the slop parameter was a key point for obtaining the best results. Several test runs were conducted to determine the optimal slop setting, and the results showed that a value of 50 consistently yielded the best performance. Therefore, this value was selected for the final configuration. A possible explanation for this result is that smaller slop values may be too restrictive, missing relevant

Table 5
Synonyms system’s performance.

Month	mAP	nDCG
2022-06	0.1150	0.2043
2022-07	0.1188	0.2055
2022-08	0.1302	0.2135
2022-09	0.1206	0.2034
2022-10	0.2026	0.3050
2022-11	0.2080	0.3088
2022-12	0.2160	0.3201
2023-01	0.2200	0.3234
2023-02	0.2000	0.2935

documents where query terms appear slightly farther apart. On the other hand, larger values might introduce too much noise, retrieving documents with weaker term associations. A slop of 50 appears to strike the right balance between flexibility and precision, allowing for meaningful proximity without overly relaxing the positional constraints. As discussed in Section 3.4.5, we explored two approaches: the first uses a *PhraseQuery* to match documents where all query terms appear in the same order as in the original query; the second extends this by incorporating additional proximity constraints based on all possible pairs and triplets of query terms, regardless of their order, using *SpanNearQuery* with `inOrder=false`. As previously discussed, in the unordered case, we restricted the proximity constraints to all possible pairs and triplets of query terms, rather than the full query, in order to avoid exceeding the *BooleanQuery* clause limit.

Table 6 compares the system’s performance when using only the full-query proximity constraints (*PhraseQuery*) against the extended approach that also includes unordered pairs and triplets (*PhraseQuery* + *SpanNearQuery*). Firstly, it can be observed that the implementation of proximity search leads to a consistent and substantial improvement in both mAP and nDCG scores compared to the baseline. In particular, the maximum change is observed in January with an increase of 7.2% for the mAP and 4.15% for the nDCG. Regarding the comparative performance of the two strategies, a trade-off emerges: relative to the *PhraseQuery*-only approach, the extended method involving unordered pairs and triplets leads to a slight decrease in mAP, while consistently improving nDCG (except for January). This suggests that incorporating unordered term combinations may retrieve more relevant documents overall, even if precision at the top ranks is marginally affected.

Table 6
Proximity Search (PS) system’s performance.

Month	PhraseQuery		PhraseQuery + SpanNearQuery	
	mAP	nDCG	mAP	nDCG
2022-06	0.1267	0.2160	0.1276	0.2189
2022-07	0.1294	0.2167	0.1293	0.2185
2022-08	0.1398	0.2237	0.1384	0.2244
2022-09	0.1304	0.2128	0.1311	0.2152
2022-10	0.2219	0.3222	0.2201	0.3227
2022-11	0.2240	0.3236	0.2226	0.3241
2022-12	0.2311	0.3342	0.2300	0.3349
2023-01	0.2393	0.3413	0.2367	0.3407
2023-02	0.2164	0.3094	0.2141	0.3099

5.7. Pseudo Relevance Feedback

We tested two different configurations of pseudo relevance feedback (PRF), aiming to evaluate the impact of different parameter settings. The first configuration, referred to as **PRF1**, is more conservative and uses the following settings: "topDocsForPRF": 10, "topTermsToAdd": 3, "originalBoost": 1.0, and "expansionBoost": 0.3. The second configuration, **PRF2**, is more aggressive and relies on "topDocsForPRF": 20, "topTermsToAdd": 5, "originalBoost": 0.8, and "expansionBoost": 0.5.

It is worth noting that PRF is typically more effective in scenarios where initial precision is high (e.g., high P@5 or P@10), as it assumes that top-ranked documents are relevant and can guide useful expansion. Therefore, evaluating PRF in isolation on a general baseline setup may not fully reflect its potential. Nevertheless, this comparison provides useful insights for tuning the PRF strategy.

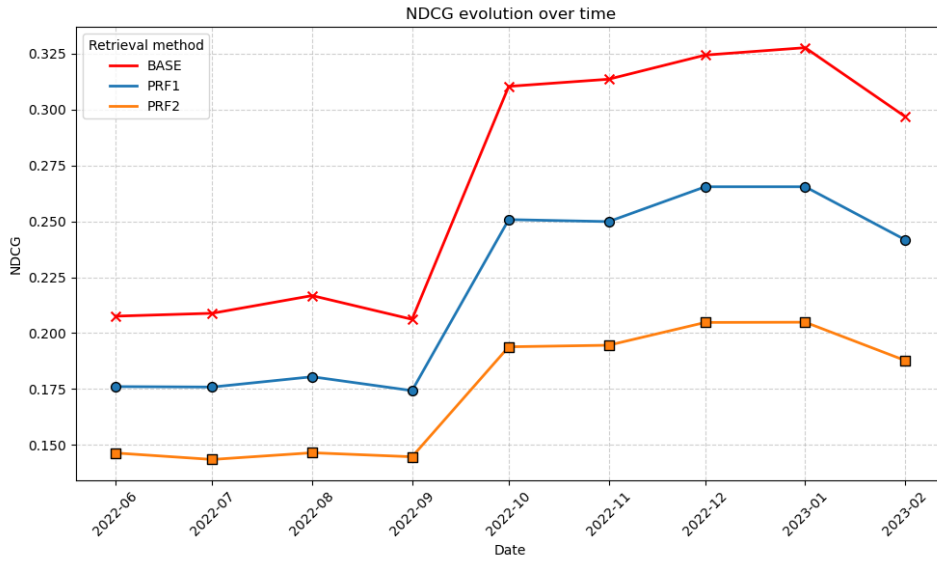


Figure 5: NDCG evolution over time for baseline, PRF1 and PRF2 configurations.

As expected, both PRF configurations resulted in a decrease in NDCG when applied on top of the baseline system. However, the comparison between PRF1 and PRF2 helps highlight the relative benefits of a more conservative versus a more aggressive expansion approach.

Later in this report, we will discuss how PRF interacts with other retrieval enhancements and whether its impact changes when combined with more effective base configurations.

5.8. Chunk Indexing

To investigate whether finer-grained indexing could enhance retrieval effectiveness, we experimented with a chunk indexing approach. Documents were split into overlapping chunks of 10 sentences each, with a shared window of 3 sentences.

As shown in Figures 6 and 7, chunk indexing consistently slightly underperforms compared to the baseline. Both mAP and NDCG values are lower across the entire time period, suggesting that splitting documents into sentence-based overlapping chunks, in this configuration, degrades retrieval performance.

In this scenario chunking probably introduces noise and dilutes term co-occurrence signals, making it harder for the system to correctly prioritize relevant documents.

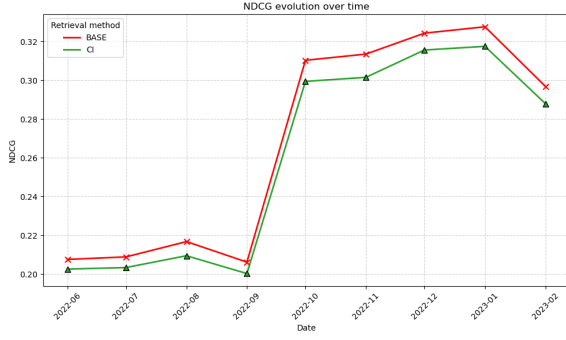


Figure 6: Chunk Indexing NDCG evolution over time

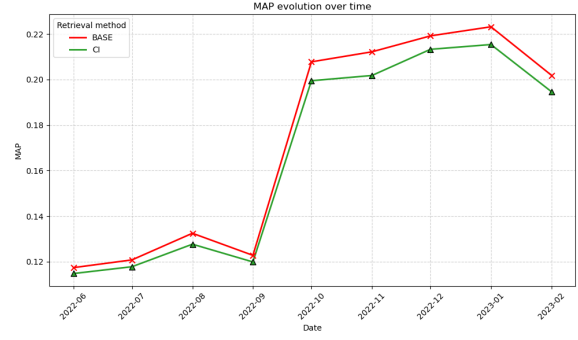


Figure 7: Chunk Indexing mAP evolution over time

5.9. Reranker

As discussed in 3.4.7, we need to determine a threshold for the number of documents to be reranked. Based on the work by Basaglia et al. [3], we observed that reranking the top 150 documents provided a good balance between efficiency and effectiveness, so we selected this value. Below is a table displaying the performance achieved by our system each month, using reranking and proximity search (approach with *PhraseQuery* + *SpanNearQuery*), which are the techniques improving our system.

Table 7

System performance with reranking and proximity search (*PhraseQuery* and *SpanNearQuery* approach). (August missing)

Month	mAP	nDCG
2022-06	0.1338	0.2255
2022-07	0.1351	0.2243
2022-08	—	—
2022-09	0.1367	0.2208
2022-10	0.2288	0.3304
2022-11	0.2339	0.3328
2022-12	0.2356	0.3403
2023-01	0.2436	0.3472
2023-02	0.2225	0.3174

Due to time constraints, we were not able to complete the run for August, as the runs involving the reranker are particularly time-consuming. This evaluation will be included in a future work.

As shown, reranking leads to improvements in both mAP and nDCG scores over the proximity search (*PhraseQuery* + *SpanNearQuery* approach) results across all months. The greatest gains are observed in November, with mAP increasing by 5.1% and nDCG by 2.7%.

5.10. Combined Strategies

In this section, we evaluate the performance of various combined approaches, all built on top of the Proximity Search method. Given the consistently strong results achieved by PS alone, it serves as the foundation for all hybrid strategies evaluated here.

We consider combinations such as PS with Chunk Indexing (PS_CI), PS with pseudo relevance feedback (PS_PRFF1), and the full combination PS_CI_PRFF1. The goal was to assess whether the integration of multiple retrieval enhancements could provide cumulative improvements.

As the figures above show, none of the combined strategies significantly outperforms the standalone PS method. In some cases, such as PS_CI_PRFF1, the addition of other techniques even slightly degrades performance. This is particularly evident in the mAP plots, where more complex configurations tend to underperform compared to PS alone.

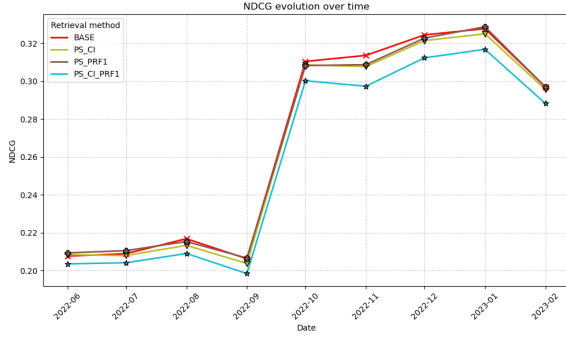


Figure 8: NDCG: baseline vs. hybrid strategies

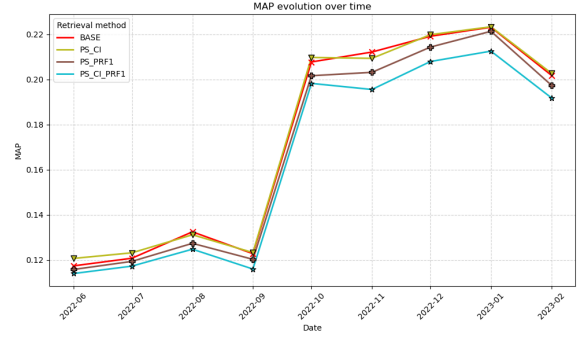


Figure 9: mAP: baseline vs. hybrid strategies

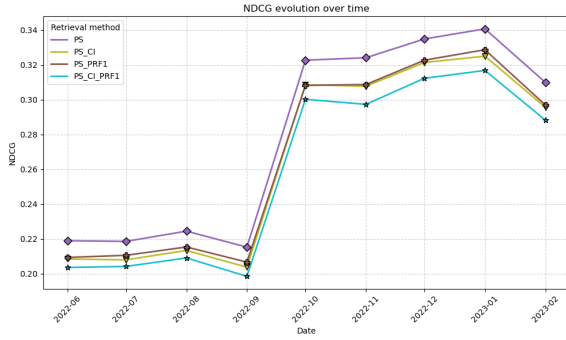


Figure 10: NDCG: PS vs. hybrid strategies

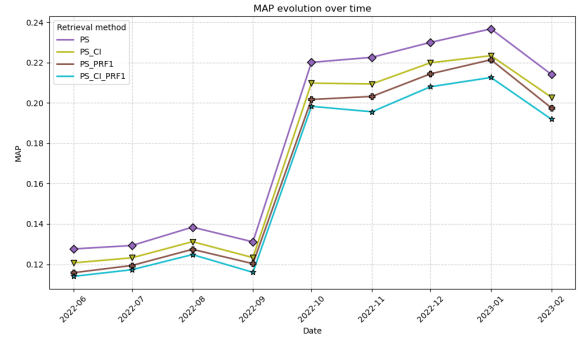


Figure 11: mAP: PS vs. hybrid strategies

One possible explanation is that pseudo relevance feedback (PRF1) may not yet be effective in this context, possibly due to insufficient initial precision. PS alone may not reach high enough early precision to make expansion terms reliably informative. Alternatively, the PRF configuration may require a different fine-tuning when applied on top of PS.

Despite this, the results demonstrate the robustness of the PS method. Even when combined with less effective strategies, PS manages to retain a high level of performance across time. This suggests that PS, as implemented, remains the most reliable and effective enhancement among those tested in this study.

5.11. Training results

In this section, we compare the best-performing configurations across various implemented techniques. These configurations are the runs we have selected for submission to the LongEval Conference:

- The best system that doesn't use reranking
- The best overall system. This is the system that achieved the best results using reranking and proximity search, *PhraseQuery* and *SpanNearQuery* approach.
- A system using chunk indexing (CI).
- A system using pseudo-relevance feedback.
- A system using proximity search (*PhraseQuery* and *SpanNearQuery* approach), chunk indexing and pseudo-relevance feedback.

Each of the systems will use the configuration of the baseline, that we recall being the one reported in Table 2. The proximity search approach is the one considering also pairs and triples (*PhraseQuery* and *SpanNearQuery*)

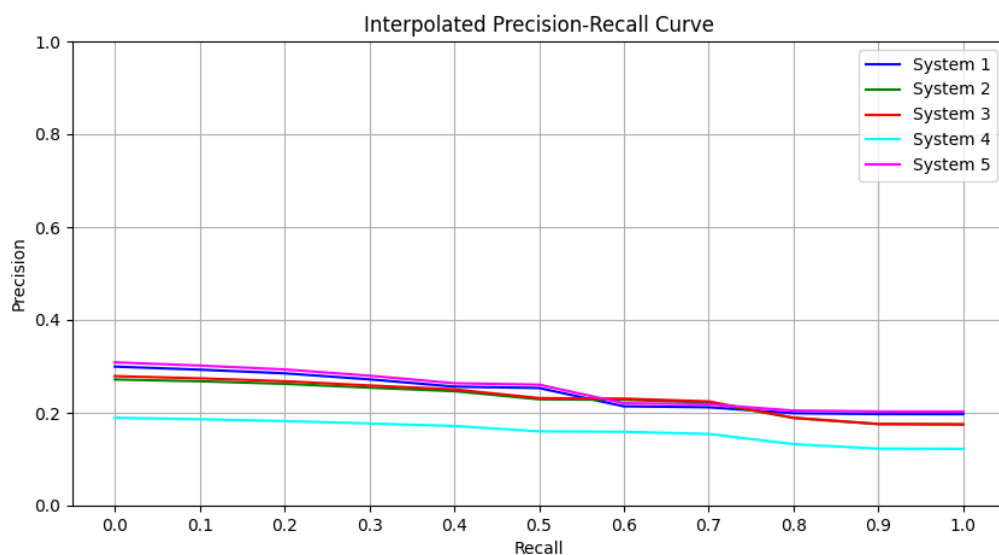
Table 8 presents, for each system configuration, the best result achieved across all months, which corresponds to the score of January.

Table 8

Performance of the submitted configurations on the training dataset

Label	System	mAP	nDCG
System 1	Baseline - Pseudo relevance feedback 1	0.1472	0.2655
System 2	Baseline - Chunk indexing	0.2154	0.3176
System 3	Baseline - Proximity search (best performing system with no reranking)	0.2367	0.3407
System 4	Baseline - Pseudo relevance feedback 1-Chunk indexing-Proximity search	0.2126	0.3168
System 5	Baseline - Proximity search-Reranking (best performing system)	0.2436	0.3472

From the results shown in Table 8, we can draw several key insights. First, proximity search alone (System 3) already provides a significant improvement over the baseline, even without reranking. However, the best overall performance is achieved by combining proximity search with reranking (System 5), which outperforms all other configurations in both mAP and nDCG, highlighting the importance of reranking in enhancing retrieval quality. Interestingly, the combination of proximity search, chunk indexing, and pseudo-relevance feedback (System 4) does not surpass the performance of simpler approaches, suggesting that the integration of multiple techniques does not necessarily lead to additive gains. Chunk indexing alone (System 2) shows a loss in performance, especially in terms of nDCG. These findings support the submission of System 3 as the best non-reranking system and System 5 as the most effective overall configuration.

**Figure 12:** Interpolated Precision Recall curves on the training dataset

The interpolated Precision-Recall curves in Figure 12 confirm the findings from the tabular metrics. System 5 consistently shows the highest precision across nearly all recall levels, reinforcing its status as the best-performing configuration overall. Systems 1, 2, and 3 follow closely, with similar trends, while System 4 clearly underperforms compared to the others. This suggests that combining multiple techniques (as in System 4) may introduce complexity without tangible performance gains, and emphasizes the effectiveness of targeted enhancements such as reranking and proximity search.

6. Conclusions and Future Work

This section summarizes the different techniques we explored in order to improve our Information Retrieval system. Starting from a basic configuration made by using Lucene's API, we tested its performance to set a reference score, and we discovered how different data can greatly change the performance of our system. We explored several possible configurations for our analyzer, including a lemmatization step of the entire collection, and we discovered that using a light stemming process produces higher score than using more complex stemmers. We moved then on with testing different approaches to improve the matching and the ranking of the documents: we first introduced a different way to index the documents, by splitting them into smaller pieces and scoring each piece individually, but we soon discovered that this approach leads our system to underperform with respect to the fixed baseline.

We later moved on into expanding the query terms using synonyms, which did not lead to an improvement of the system, to the releasing of terms' positions in the proximity search approach, which instead proved to increase the effectiveness of the system, especially when fine-tuning the slop parameter. We also explored the possibility of using the top-most retrieved documents to improve the query itself through the pseudo relevance feedback technique, especially useful when precision is high at the top-most ranking positions. Finally, we used sentence embeddings to rerank the results returned by our system through the Roberta-Large model, which again leads us to a better improvement of our system's performance.

When all these techniques are deployed individually, it may appear difficult to understand if the system is improving, however combining them all appears not to be the most effective way to increase the performance of the system, which is also very sensitive to data that are fed as input. In order to continue improving the efficiency of an IR system, further research is needed. As regards synonyms and query expansion, the most intuitive way to improve the query is to use a language model to learn the synonyms directly by the collection, instead of relying on external datasets. As regards the lemmatization process, it may be useful to construct a dictionary of arguments and use the same dictionary to tag each provided query with a more general word, and use those to improve the pool of matched documents. Another possible improvement of the system could be made by joining together similar queries, so that the user may find more documents related to the possibly imprecise query he submitted. Once again, the huge development of LLMs that we are seeing nowadays can be a great tool to improve the effectiveness of those IR systems, especially when the data collections are mostly made of textual documents.

7. Group Members Contribution

Andrea Bruttomesso In the early stages of development helped with the overall design of the SystemRunner class and organized part of the codebase, built the WebDocument Parser and contributed to improve the Indexing stage. Later on designed and implemented the Chunk Indexing and Pseudo Relevance Feedback strategies, fine tuning and running experiments with them. During the later stages focused on running many different experiments with the system, collecting, analyzing and representing the results reported in the final document.

Daniele Cavazza Implementation of WebDocument class. Addition of new stemming technique. Basic implementation of Searcher and subsequent adaptation of the pipeline in SystemRunner class. Implementation of MultiThreadedSearcher. Implementation of Flask Server, HttpHelper and other adaptations to run reranking. After the selection of the best performing model: runs of the system with configuration Proximity Search + ReRanking. Synonyms, ReRanking sections and brief description of the Experimental Setup on report.

Alessandro Corrà Implementation of the WebAnalyzer class and related classes. Adaptation of the SystemRunner class to support the new analyzer configurations. Execution of multiple experimental runs to evaluate the impact of different analyzer settings and variations of the BM25 scoring function. Collaboration in the execution and evaluation of synonym-based experiments. General setup of the report. Introduction, parts of the Methodology section, and contributions to the Results section with analysis of experimental outcomes.

Simone Peraro Improving efficiency on WebDocument parsing phase. Implementing base version of multi threaded indexer. Implement parser for QualityQueries. Implement support class for inspecting the index. Implement utility to filter duplicate qrels. Initial implementation of the main system runner. Little refactor for searcher constructor. Experimenting with OpenNLP. Experimenting with collection lemmatizing, and corresponding runs simulations. Baseline run simulations. Basic notebook to plot evaluation measures. Report contributions.

Davide Seghetto Code: implementation of all the classes for the indexer, excluded the multi-thread version; searcher, in particular, query expansion with synonyms, fuzzy query, proximity search (both approaches) and reranker; AnalyzerParams and SearcherParams. Report: related works; first part of methodology; implementation and results for fuzzy search, proximity search and reranker and training results.

References

- [1] Sushilkumar Chavhan, M. Raghuwanshi, R. Dharmik, Information Retrieval using Machine learning for Ranking: A Review , <https://iopscience.iop.org/article/10.1088/1742-6596/1913/1/012150/meta>, 2021.
- [2] R. Nogueira, W. Yang, K. Cho, J. Lin, Multi-stage document ranking with bert, 2019. URL: <https://arxiv.org/abs/1910.14424>. arXiv: 1910.14424.
- [3] A. Basaglia, A. Stocco, M. Popovic, N. Ferro, Seupd@clef:team dam on reranking using sentence embedders, 2024. URL: <https://ceur-ws.org/Vol-3740/paper-214.pdf>.
- [4] R. Pramana, Debora, J. J. Subroto, A. A. S. Gunawan, Anderies, Systematic literature review of stemming and lemmatization performance for sentence similarity, in: 2022 IEEE 7th International Conference on Information Technology and Digital Applications (ICITDA), IEEE, 2022, pp. 1–6. doi:10.1109/icitda55840.2022.9971451.
- [5] V. Balakrishnan, E. Lloyd-Yemoh, Stemming and lemmatization: A comparison of retrieval performances, 2014. URL: <https://api.semanticscholar.org/CorpusID:52998253>.
- [6] K. Abedini, A. Akysh, A. Fahoud, Seupd@clef: Team kalu on improving search engine performance with query expansion and re-ranking approach, 2024. URL: <https://ceur-ws.org/Vol-3740/paper-214.pdf>.
- [7] I. A. H. Chen, J. Moncada-Ramírez, N. Santini, G. Zago, N. Ferro, Seupd@clef: Team jhiuming on enhancing search engine performance with character n-grams, query expansion, and named entity recognition, 2023. URL: <https://ceur-ws.org/Vol-3497/paper-185.pdf>.
- [8] The Apache Software Foundation, Apache Lucene, <https://lucene.apache.org/>, 2025.
- [9] The Apache Software Foundation, Apache OpenNLP, <https://opennlp.apache.org/>, 2025.
- [10] B. Sagot, The lefff, a freely available and large-coverage morphological and syntactic lexicon for french, 7th international conference on Language Resources and Evaluation (LREC 2010) (2010).
- [11] Coulomb, Claude, A French Lemmatizer in Python based on the LEFFF, <https://github.com/ClaudeCoulombe/FrenchLefffLemmatizer>, 2017.
- [12] W. Yin, H. Schütze, Multigrancnn: An architecture for general matching of text chunks on multiple levels of granularity, in: Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics (ACL), Association for Computational Linguistics, 2015, pp. 63–73.
- [13] B. Sagot, D. Fišer, Building a free french wordnet from multilingual resources, 2008.
- [14] N. Reimers, I. Gurevych, Sentence-bert: Sentence embeddings using siamese bert-networks, 2019. URL: <https://aclanthology.org/D19-1410>.
- [15] N. Reimers, I. Gurevych, sentence-transformers/all-roberta-large-v1, <https://huggingface.co/sentence-transformers/all-roberta-large-v1>, 2020.
- [16] Pallets Projects, Flask documentation, <https://flask.palletsprojects.com/>, 2023. Accessed: 2025-05-05.